



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Multi-algorithm path exploration and navigation simulator

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements for the
Course of*

CSA0609 - DESIGN AND ANALYSIS OF ALGORITHMS
to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

G Uday Siva Sai (192425259)

Under the Supervision of

Dr. Solainayagi

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Multi-algorithm path exploration and navigation simulator” has been carried out by G Uday Siva Sai (192425259) under the supervision of Dr. Solainayagi and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech Artificial Intelligence And Machine Learning program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

**Dr. Dr. F. Mary Harin Fernandez
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.**

COURSE FACULTY

**Dr. Solainayagi
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.**

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We, **Deepak V (192411021)**, **Champak Rupesh K (192425123)**, **G. Uday Siva Sai (192425259)**, **R. Uday Kumar Reddy (192424190)** and **N. MadhuSudhan (192425303)** of the AI&DS, AI&ML, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**Multi-Algorithm Path Exploration and Navigation Simulator**” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names

Deepak V (192411021)
Champak Rupesh K (192425123)
G. Uday Siva Sai (192425259)
R. Uday Kumar Reddy (192424190)
N. MadhuSudhan (192425303)

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. %
Deepak V (192411021)	Requirement analysis and project planning	Designed graph structure and navigation modules	Tested graph inputs and path generation	Abstract, Introduction	20%
Champak Rupesh K (192425123)	Algorithm implementation	Implemented BFS and DFS algorithms	Verified algorithm outputs	Literature Review, Algorithms	20%
G. Uday Siva Sai (192425259)	Algorithm development and integration	Implemented Dijkstra and A* algorithms	Compared path distances and explored nodes	Methodology, Implementation	20%
R. Uday Kumar Reddy (192424190)	System coordination and visualization	Developed system architecture and result visualization	Tested complete system and analyzed results	Results, Discussion	20%
N. MadhuSudhan (192425303)	Documentation and system support	Assisted module integration and interface development	Performed test cases and performance analysis	Conclusion, Future Work	20%
Total					100%

ABSTRACT

Navigation and path-finding are important engineering problems in applications such as campus navigation, transportation networks, communication networks, and autonomous systems. Finding an efficient route between a source and destination becomes challenging when a network contains multiple interconnected locations with different path costs. Different path-finding algorithms use different search strategies and therefore may produce different exploration patterns and computational performance. A system is required to analyse these differences and identify suitable algorithms for different navigation conditions.

The Multi-Algorithm Path Exploration and Navigation Simulator is proposed to provide a graph-based environment for implementing, testing, and comparing multiple path-finding algorithms. The system represents locations as nodes and connections as weighted or unweighted edges. It implements Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra's algorithm, and A* to explore routes between selected source and destination nodes.

The methodology involves graph construction, source and destination selection, algorithm execution, path reconstruction, and performance evaluation. Each algorithm is tested using common navigation scenarios, and its performance is evaluated using path distance, number of explored nodes, and execution time. A visualization component is also used to represent the navigation graph, explored nodes, and generated route. This provides a consistent environment for analysing the behaviour of different search techniques.

The experimental results demonstrate that Dijkstra's algorithm and A* can identify minimum-cost routes in the tested weighted network, while BFS is effective for unweighted path exploration. DFS successfully explores the network but does not guarantee a minimum-cost route. A* can reduce unnecessary exploration when an appropriate heuristic is available. The measured execution times were within the specified response requirement for the test network.

The project concludes that algorithm selection has a significant effect on route quality and search efficiency. The developed simulator provides a practical and educational engineering solution for understanding, testing, and comparing graph-based navigation algorithms under controlled conditions.

Keywords: *Graph Theory, Path Finding, BFS, DFS, Dijkstra Algorithm, A* Search*

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	Certificate from the Supervisor	2
ii	Declaration by the Candidate	3
iii	Individual Contribution Statement	4
iv	Abstract	5
v	List of Figures	7
vi	List of Tables	8
vii	List of Abbreviations	9
1	CHAPTER 1: Introduction and Engineering Problem	10
2	CHAPTER 2: Literature Review and Existing Solutions	13
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	15
4	CHAPTER 4: Implementation, Testing & Results, Project Management	21
5	CHAPTER 5: Conclusion, Future Work and Reflection	28
	References	30
	Appendices	31
	Capstone Project Outcome Mapping Sheet	32

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	System Architecture of the Multi-Algorithm Path Exploration and Navigation Simulator	18
4.1	Dashboard of the Multi-Algorithm Path Exploration and Navigation Simulator	22
4.2	Interactive Graph Simulator showing graph traversal and algorithm results	22

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative Analysis of Existing Path-Finding Approaches	14
3.1	Stakeholder / User Requirements	15
3.2	Functional and Non-Functional Requirements	16
3.3	Constraints and Applicable Standards	16
3.4	Design Specifications	16
3.5	Evaluation Matrix for Alternative Solutions	17
3.6	Trade-off Analysis	19
3.7	Sustainability, Ethics, Safety and Security Considerations	20
3.8	Risk Register	20
4.1	Tools / Software Used and Purpose	21
4.2	Test Plan and Verification	23
4.3	Specification vs. Achieved Values	23
4.4	Representative Simulation Results (Sample Run)	24
4.5	Achievement of Objectives	25
4.6	Roles and Responsibilities	26
4.7	Estimated Project Cost	27

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
DAA	Design and Analysis of Algorithms	–
UI	User Interface	–
UX	User Experience	–
DP	Dynamic Programming	–
V	Number of Vertices (Nodes) in a Graph	–
E	Number of Edges in a Graph	–
O()	Big-O Notation (Asymptotic Time Complexity)	–
ms	Milliseconds	time

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Shortest-path computation is one of the most fundamental and widely applied problems in the Design and Analysis of Algorithms (DAA). It underlies technologies that are used on a daily basis, from vehicle navigation and public-transport routing to network packet routing and logistics planning. Over several decades of algorithmic research, a number of classical graph algorithms have been developed to solve variants of the shortest-path problem, each with different assumptions, guarantees and computational trade-offs. Dijkstra's Algorithm greedily expands the nearest unvisited vertex and is efficient for graphs with non-negative edge weights. A* Search extends this idea using a heuristic estimate of the remaining distance to the destination, allowing it to reach a target faster on suitable graphs. Bellman-Ford repeatedly relaxes all edges and can correctly handle negative edge weights and detect negative-weight cycles, at a higher computational cost. Floyd-Warshall applies dynamic programming to compute shortest paths between every pair of vertices simultaneously. In a typical DAA course, these algorithms are usually taught through pseudocode and manually traced examples on paper, which makes it difficult for a learner to develop an intuitive, comparative understanding of how each algorithm actually behaves on the same underlying network.

1.2 Need for the Project

Commercial navigation applications compute and display a route to the user but intentionally hide the underlying algorithm and its internal decision process, since their objective is only to deliver a usable route rather than to teach algorithmic behaviour. As a result:

- Students and self-learners of DAA have no easy way to see, step by step, how a shortest-path algorithm explores a graph.
- There is no single platform where multiple shortest-path algorithms can be executed on the identical graph and compared directly, under the same conditions.
- The effect of graph structure (density, presence of negative weights, number of nodes) on algorithm behaviour and performance is difficult to observe without repeated manual tracing.
- Existing classroom tools are largely static (slides, textbook diagrams) and do not provide an interactive, hands-on simulation environment.

These limitations directly affect engineering education outcomes, since a learner who cannot observe algorithmic behaviour in an interactive setting is less likely to develop strong intuition for algorithm selection, complexity trade-offs, and correctness in real engineering scenarios such as routing and network design. There is, therefore, a genuine need for an interactive graph-based navigation simulator that unifies graph creation, algorithm execution, visualization and performance comparison on one platform.

1.3 Problem Statement

Existing navigation systems provide routes to end users but generally do not expose the algorithmic process behind route selection, and students lack an interactive environment where DAA graph algorithms can be executed, visualized, and compared in real time on the same network. The engineering problem addressed by this project is the design and development of a single interactive platform that supports graph creation, route (source/destination) selection, execution of multiple shortest-path algorithms, step-by-step visualization of their traversal, and quantitative performance comparison, while remaining usable by students with only an introductory understanding of graph algorithms.

1.4 Project Aim

The aim of this project is to design and develop an interactive, web-based, graph-based navigation simulator that allows a user to build a weighted graph, run multiple classical shortest-path algorithms (Dijkstra, A*, Bellman-Ford and Floyd-Warshall) on that graph, visualize each algorithm's traversal, and compare their performance using common quantitative metrics, thereby improving the practical understanding of the Design and Analysis of Algorithms.

1.5 Project Objectives

- To design and develop an interactive graph-based navigation system supporting dynamic creation and editing of weighted graphs.
- To implement multiple classical shortest-path algorithms – Dijkstra, A* Search, Bellman-Ford and Floyd-Warshall – within a common execution engine.
- To evaluate and compare algorithm performance using common metrics such as execution time, shortest-path distance, number of nodes explored and relative memory/space utilization.
- To visually display computed shortest routes and animate the step-by-step traversal of each algorithm on the graph.
- To generate downloadable, structured reports summarising graph configuration, algorithm results and comparative performance, thereby improving practical understanding of the Design and Analysis of Algorithms.

1.6 Scope

Included in Scope

- Interactive creation and editing of weighted, directed or undirected graphs representing a road/location network.
- Source and destination (route) selection with input validation.
- Implementation and execution of Dijkstra, A*, Bellman-Ford and Floyd-Warshall algorithms.
- Step-by-step animated visualization of algorithm traversal on the graph.
- Computation and tabulation of comparative performance metrics across algorithms.
- Generation of downloadable simulation/comparison reports, together with sample and randomly generated demonstration graphs (“Demo Mode”).

Excluded from Scope

- Integration with real-world live traffic, GPS or mapping data (the graphs used are user-created or sample/demo networks, not live geospatial data).
- Turn-by-turn voice navigation or mobile-device GPS tracking.
- Algorithms other than the four shortest-path algorithms explicitly listed above (e.g., k-shortest paths, vehicle routing with capacity constraints).

Assumptions and Boundary Conditions

- Edge weights represent an abstract distance/cost metric (for example, road distance) entered by the user.
- For Dijkstra's Algorithm and A* Search, graphs are assumed to have non-negative edge weights, consistent with the theoretical requirements of these algorithms.
- The number of nodes/edges handled by the simulator is bounded by what can be reasonably rendered and animated in a browser (typically tens of nodes for interactive use, as reflected in the provided sample graphs).

1.7 Expected Engineering Outcome

The expected engineering outcome of this project is a working software prototype: a browser-based, interactive graph navigation simulator (algorithm explorer) consisting of a dashboard, a graph-editing simulator, a multi-algorithm path-finding engine, a performance-comparison module and a reporting module. The system is expected to function as a teaching and demonstration tool that makes the internal behaviour of shortest-path algorithms observable, comparable and reproducible for students of the Design and Analysis of Algorithms.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review of existing work for this project was carried out by examining three categories of prior art: (i) the standard algorithmic treatment of shortest-path problems in established algorithms literature, most notably Cormen, Leiserson, Rivest and Stein's "Introduction to Algorithms", which is the standard reference for Dijkstra's, Bellman-Ford's and Floyd-Warshall's algorithms; (ii) widely used commercial and consumer navigation/mapping applications, considered from the perspective of what routing functionality they expose to an end user; and (iii) existing classroom and web-based algorithm-visualisation tools that are commonly used to teach graph algorithms. This review was used to identify the specific gap that this project's simulator is intended to fill.

2.2 Review of Existing Work

Classical shortest-path algorithms are extensively documented in standard algorithms literature. Dijkstra's Algorithm is established as an efficient greedy method for single-source shortest paths on graphs with non-negative weights, with an implementation using a priority queue achieving $O((V + E) \log V)$ time complexity. A* Search is a well-known extension of best-first search that uses an admissible heuristic to guide exploration toward the destination, making it particularly effective for point-to-point routing when a good heuristic (such as straight-line distance) is available. The Bellman-Ford algorithm is documented as a dynamic-programming style relaxation method capable of handling negative edge weights and detecting negative-weight cycles, at the cost of a higher $O(V \cdot E)$ time complexity compared to Dijkstra's Algorithm. The Floyd-Warshall algorithm is a classical dynamic-programming approach for computing all-pairs shortest paths in a single $O(V^3)$ procedure, which is useful when shortest distances between every pair of nodes are required rather than a single source-destination path.

Commercial navigation and mapping applications are, in practice, the most visible real-world application of shortest-path algorithms, but they are engineered as closed, production systems: they return a computed route to the user without exposing which algorithm or heuristic produced it, and they are not designed to support side-by-side comparison of multiple algorithms on a user-defined graph. Separately, a number of web-based algorithm-visualisation tools exist in the educational space, typically allowing a learner to watch a single classical algorithm execute step by step on a predefined or small example graph. These tools are valuable for building intuition about one algorithm at a time but generally do not allow a learner to build a custom weighted graph and then run several different shortest-path algorithms on that same graph to directly compare their behaviour and performance.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
-------------------	------------	-------------	----------------------------

Commercial navigation applications (e.g., mainstream map/GPS apps)	Highly optimised, production-grade routing; real-time traffic integration; mass usability.	Underlying algorithm and decision process are hidden from the user; not designed for teaching or algorithm comparison.	Motivates the need for a transparent, educational alternative that exposes algorithmic behaviour.
Static DAA visualization tools (e.g., algorithm-visualisation websites used in classrooms)	Provide step-by-step visual traces for a single algorithm at a time; widely used as teaching aids.	Typically demonstrate one algorithm on a fixed example graph; limited support for user-defined graphs or direct multi-algorithm comparison on the same network.	Establishes the value of visualization for learning, while highlighting the gap this project fills – multi-algorithm comparison on a common, user-editable graph.
Manual pseudocode tracing (textbook / classroom method)	No software dependency; reinforces step-by-step algorithmic thinking when done carefully.	Time-consuming, error-prone for larger graphs, and does not scale to comparing four algorithms or to graphs with more than a handful of nodes.	Confirms the practical need for an automated, interactive simulation environment for larger or repeated experiments.

Table 2.1: Comparative Analysis of Existing Path-Finding Approaches

2.4 Research / Engineering Gap

Based on the review above, the identified gap is the absence of a single, interactive, browser-based platform that (a) allows a user to freely create or load a weighted graph, (b) executes multiple classical shortest-path algorithms – Dijkstra, A*, Bellman-Ford and Floyd-Warshall – on that exact same graph, (c) visualises the step-by-step traversal of each algorithm, and (d) presents a direct, quantitative, side-by-side performance comparison across all executed algorithms. Existing commercial systems address routing but hide the algorithm; existing visualisation tools address teaching but typically cover one algorithm on a fixed graph at a time. Neither category combines user-driven graph creation with multi-algorithm, comparative, visual execution.

2.5 Proposed Contribution

This project contributes a unified Multi-Algorithm Path Exploration and Navigation Simulator that directly addresses the identified gap. Unlike commercial navigation systems, the simulator exposes the full algorithmic process – including step-by-step traversal, relaxation counts and adjacency information – to the user. Unlike single-algorithm visualisation tools, it allows Dijkstra, A*, Bellman-Ford and Floyd-Warshall to be executed on the identical, user-defined or sample graph within one session, and it automatically tabulates comparative performance metrics (execution time, path distance, nodes explored/relaxations) for all four algorithms

together, in addition to supporting graph creation, saved graphs, a dedicated algorithm explorer, and downloadable reports.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
DAA Student / Learner	Ability to build or load a weighted graph and observe, step by step, how different shortest-path algorithms explore it.
Course Instructor	A demonstration tool that can be used in lectures/labs to compare algorithm behaviour on the same graph in real time.
Project Evaluator / Assessor	Clear, exportable evidence of correct algorithm execution and measurable performance comparison for grading.
General End User (self-learner)	An intuitive, browser-based interface that requires no installation and works with sample graphs (Demo Mode) if the user does not want to build one from scratch.

Table 3.1: Stakeholder / User Requirements

Functional & Non-Functional Requirements

Requirement Type	Description	Measurable Target
Functional	Create/edit graph vertices and weighted edges	Add/edit/delete a node or edge in ≤ 3 user actions
Functional	Select source and destination nodes with validation	Invalid/disconnected selections rejected with a clear message
Functional	Execute Dijkstra, A*, Bellman-Ford and Floyd-Warshall on the same graph	All 4 algorithms return a result for any valid graph within their supported weight range
Functional	Animate step-by-step traversal for each algorithm	Playback with variable speed (e.g., 1x) and step navigation
Functional	Generate a downloadable comparison report	Report exported in ≤ 2 user actions

Functional	Track relative memory/space utilization alongside time and distance for each algorithm	Performance Comparison module records a memory/space indicator per algorithm run
Non-Functional	Usability – no installation, browser-based access	Runs in a standard modern web browser
Non-Functional	Responsiveness of algorithm execution for demo-scale graphs	Sub-second execution time for demo-scale graphs ($\leq \sim 10$ nodes), as observed in Table 4.4
Non-Functional	Learnability	A first-time user can complete the 5-step Quick Start workflow without external help

Table 3.2: Functional and Non-Functional Requirements

3.2 Constraints & Applicable Standards

As a browser-based educational software project (with no physical hardware, chemical, or safety-critical component), the applicable constraints are primarily technical and usability-related rather than regulatory. The constraints and standards considered relevant to this project are listed below; categories such as environmental or health-and-safety engineering standards were assessed and found not applicable to a purely software-based simulator.

Standard / Guideline	Requirement	How Applied in This Project
W3C Web Content Accessibility Guidelines (WCAG) 2.1	Interfaces should be perceivable and operable for a wide range of users.	Considered for colour-contrast and control-labelling choices in the dashboard and simulator UI.
IEEE Citation Reference Guide	Consistent, traceable citation of algorithmic and technical sources.	Adopted as the referencing style for this report (see References section).
Institutional academic integrity policy	Original work; disclosure of tools/AI assistance used.	Declared in the Declaration section of this report.
Technical constraint – algorithm applicability	Dijkstra and A* require non-negative edge weights to guarantee correctness.	Enforced through graph input validation before running these two algorithms.

Table 3.3: Constraints and Applicable Standards

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Number of supported algorithms	4 (Dijkstra, A*, Bellman-Ford, Floyd-Warshall)	count	High
Graph editing operations	Add node, add edge, delete, select	–	High
Demo graph size (sample)	8 nodes, 12 edges (Sample City Network)	nodes/edges	Medium
Comparative metrics reported per algorithm	Execution time, path distance, nodes explored/relaxations	–	High
Playback control for traversal animation	Step navigation and adjustable playback speed (e.g., 1x)	–	Medium
Report export	Downloadable report of graph, algorithm results and comparison	–	Medium

Table 3.4: Design Specifications

3.4 Alternative Solutions & Evaluation

Alternative 1 – Single-Algorithm Visualizer: A simplified system that implements and visualises only one shortest-path algorithm (e.g., Dijkstra) at a time, similar to many existing classroom visualisation tools. This alternative is simpler to build and validate but does not allow direct, same-graph comparison between algorithms.

Alternative 2 – Multi-Algorithm Simulator with Comparison (Selected Design): A system that implements all four target algorithms within a common engine, runs them on the identical graph, and presents both individual visual traversal and a consolidated comparison table/chart, as described in this report.

Alternative 3 – Offline Desktop / Command-Line Tool: A non-graphical or desktop-only tool that computes shortest paths and prints numerical results without an interactive, animated visualization layer.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Educational value (multi-algorithm comparison)	High	Low	High	Low
Usability / interactivity	High	Medium	High	Low
Development complexity	Medium	Low	Medium–High	Low

Accessibility (no installation)	Medium	High	High	Low
Reliability of comparison	High	N/A (single algorithm)	High (same-graph comparison)	Medium

Table 3.5: Evaluation Matrix for Alternative Solutions

3.5 Selected Design & Detailed Design

Alternative 2 – the Multi-Algorithm Simulator with Comparison – was selected because it uniquely satisfies the high-weight criteria of educational value and reliability of comparison identified in Table 3.5, directly fulfils the problem statement in Section 1.3 (a single platform for execution, visualization and comparison), and remains browser-accessible with no installation, unlike Alternative 3. The higher development complexity relative to Alternative 1 was accepted as a justified trade-off because it is the only option that allows same-graph, side-by-side comparison of all four target algorithms, which is the project's core educational contribution.

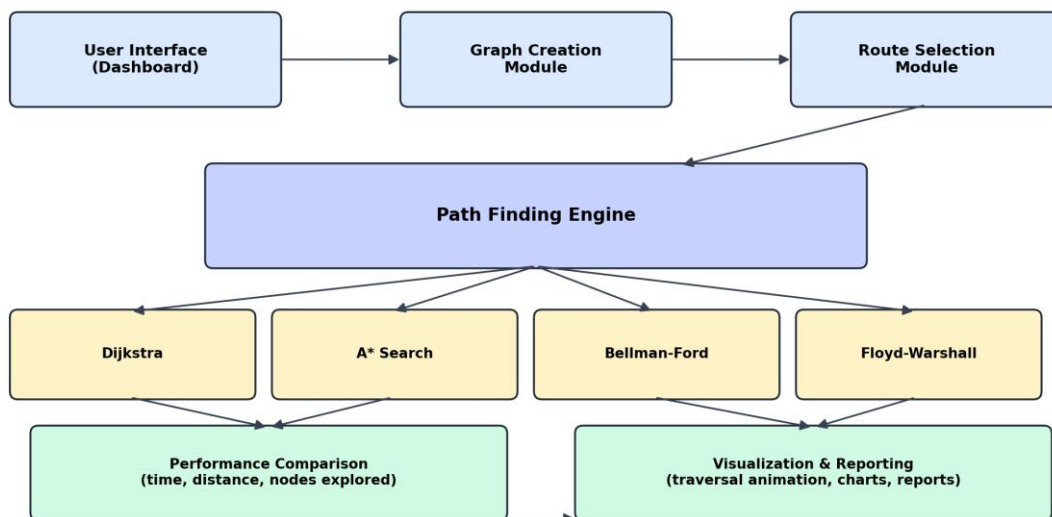


Figure 3.1: System Architecture of the Multi-Algorithm Path Exploration and Navigation Simulator

As shown in Figure 3.1, the selected design follows a modular pipeline. The User Interface (Dashboard) provides entry points into the Graph Creation module, where vertices and weighted edges are defined or loaded from a sample graph. The Route Selection module validates the chosen source and destination before handing the graph data to the central Path Finding Engine, which dispatches execution to the four algorithm implementations – Dijkstra, A* Search, Bellman-Ford and Floyd-Warshall. Each algorithm returns its computed path, distance and step trace to two downstream modules: the Performance Comparison module, which tabulates execution time, path distance and nodes explored across all four algorithms,

and the Visualization and Reporting module, which animates the traversal on the graph canvas and compiles a downloadable report.

A key engineering calculation underlying the Path Finding Engine is the standard asymptotic time complexity of each algorithm, which determines the relative cost of running all four on a graph with V vertices and E edges: Dijkstra's Algorithm (with a priority queue) runs in $O((V + E) \log V)$; A* Search runs in a complexity that depends on the quality of the heuristic used, but is bounded by the same order as Dijkstra's Algorithm in the worst case; Bellman-Ford runs in $O(V \cdot E)$; and Floyd-Warshall runs in $O(V^3)$. These complexity relationships, which are also summarised on the system's own "Algorithm Overview" dashboard panel (Figure 4.1), directly informed the decision to let the Performance Comparison module measure actual execution time and nodes explored rather than relying on asymptotic complexity alone, since real, small-graph performance can differ from the worst-case order shown.

At the systems level, the five modules in Figure 3.1 interface through a shared in-memory graph data structure: the Graph Creation and Route Selection modules populate this structure, the Path Finding Engine consumes it (without modifying the user's original graph) and produces per-algorithm result objects, and both downstream modules (Performance Comparison, Visualization and Reporting) consume those result objects independently, allowing the comparison table and the animated visualisation to always stay consistent with one another for a given run.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Central multi-algorithm engine vs. separate per-algorithm modules	Fully separate, independent modules per algorithm	A shared engine keeps graph state and result format consistent across all four algorithms, simplifying fair comparison	Slightly higher initial design complexity to define a common interface	Shared engine adopted – required for consistent, same-graph comparison (Section 3.5)
Client-side (browser) execution vs. server-side execution	Server-side algorithm execution with client only for display	No server round-trip; instant feedback; simpler deployment for a teaching tool	Execution speed depends on the user's own device	Client-side execution adopted – matches the no-installation, instant-feedback usability requirement (Table 3.2)
Step-by-step animated traversal vs.	Show only the final shortest path and distance	Animated traversal directly supports the	Requires additional development	Animated traversal adopted – core to the

final-result-only display		project's educational aim of exposing algorithmic decisions (Section 1.4)	effort to record and replay each algorithm's step trace	project's problem statement (Section 1.3)
Support for negative weights (Bellman-Ford) vs. non-negative-only design	Restrict the whole simulator to non-negative weights only	Including Bellman-Ford broadens educational coverage to negative-weight and cycle-detection scenarios	Requires input validation logic to selectively allow/disallow algorithms based on graph weights	Selective validation adopted – Dijkstra/A* restricted to non-negative graphs; Bellman-Ford/Floyd-Warshall allowed to handle negative weights

Table 3.6: Trade-off Analysis

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Being a purely software, browser-based tool, the system requires no physical materials and consumes only ordinary computing energy.	Evaluated the option of a lightweight, client-side implementation vs. a server-heavy architecture requiring always-on backend infrastructure.	Client-side execution was prioritised in Section 3.6, reducing continuous server energy consumption and hosting overhead.
Ethics	The system is an educational tool; it must accurately represent algorithm behaviour and must not misrepresent computed shortest paths or performance figures to the user.	Evaluated whether displayed results (distance, execution time, nodes explored) are generated live from the actual algorithm run rather than hard-coded or simulated values.	Design requires live, on-the-fly computation of all displayed metrics for every run, preserving the tool's academic honesty and reliability as a teaching aid.
Health & Safety	As a desk-based, browser-only application, there is no physical health-and-safety hazard	Considered general screen-based usage guidance (e.g., avoiding excessively long, uninterrupted sessions) as a general good-	No design change required; noted as a general usage recommendation only.

	associated with using the system.	practice note rather than a project-specific risk.	
Security	The system may accept user-provided graph data (nodes, edge weights) and, if extended to persistence, saved graphs.	Evaluated the need for input validation on graph data (e.g., rejecting malformed weights, preventing script injection through node/edge labels).	Input validation was added as a functional requirement (Table 3.2 – Route Selection validation) covering both algorithmic correctness and basic input-sanitisation concerns.

Table 3.7: Sustainability, Ethics, Safety and Security Considerations

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect algorithm implementation producing wrong shortest path	Medium	High	High	Cross-validate each algorithm's output against manually computed results on small test graphs before integrating into the comparison engine.	Low
Running Dijkstra/A* on a graph containing negative edge weights	Medium	Medium	Medium	Input validation restricts Dijkstra and A* to non-negative-weight graphs; Bellman-Ford/Floyd-Warshall offered for negative-weight cases.	Low
Performance degradation / UI freezing on very large user-created graphs	Low	Medium	Medium	Bound the recommended graph size for interactive use (Section 1.6) and provide pre-sized sample graphs (e.g., Sample City Network, Large Random Graph).	Low

Misinterpretation of comparison results by learners (e.g., assuming worst-case complexity always matches observed time)	Medium	Low	Low–Medium	Display both the algorithm's theoretical complexity (Algorithm Overview panel) and the measured run-time together, with contextual labelling.	Low
---	--------	-----	------------	---	-----

Table 3.8: Risk Register

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed using an iterative, module-by-module approach consistent with the architecture in Figure 3.1: the Graph Creation and Route Selection modules were implemented first to establish the core data structures, followed by the four algorithm implementations within the Path Finding Engine, and finally the Performance Comparison and Visualization/Reporting modules were layered on top of the working engine. Each module was manually tested against small, hand-traceable graphs before being integrated with the next module, so that correctness of the underlying algorithms was established before the animation and comparison layers were added. The primary resource used was the set of sample graphs bundled with the system (Sample City Network, Small Road Network, Campus Navigation, City Network, Dense Graph, and Large Random Graph), which provided a consistent, repeatable basis for development testing and for the demonstration results reported in Section 4.4.

4.2 Software Implementation & Modern Tools Used

The Multi-Algorithm Path Exploration and Navigation Simulator is implemented as a client-side web application, so that it can run directly in a standard browser with no installation, consistent with the non-functional usability requirement in Table 3.2. The graph is held as an in-memory adjacency data structure (visible to the user as the Adjacency panel in Figure 4.2); this structure is shared by all four algorithm implementations so that they operate on an identical input for every comparison run.

Tool / Technology	Purpose	Stage Used
HTML5	Structure of the dashboard, graph simulator and reports pages	Front-end development
CSS3 (custom dark-theme styling)	Visual styling of the dashboard, cards, and graph canvas	Front-end development
JavaScript (client-side scripting)	Graph data structures, algorithm implementations, DOM interactivity and animation control	Core application logic
SVG / Canvas-based graph rendering	Drawing nodes, weighted edges and highlighting the animated traversal path	Visualization module

Browser Developer Tools	Debugging, performance inspection and UI verification during development	Testing and debugging
-------------------------	--	-----------------------

Table 4.1: Tools / Software Used and Purpose

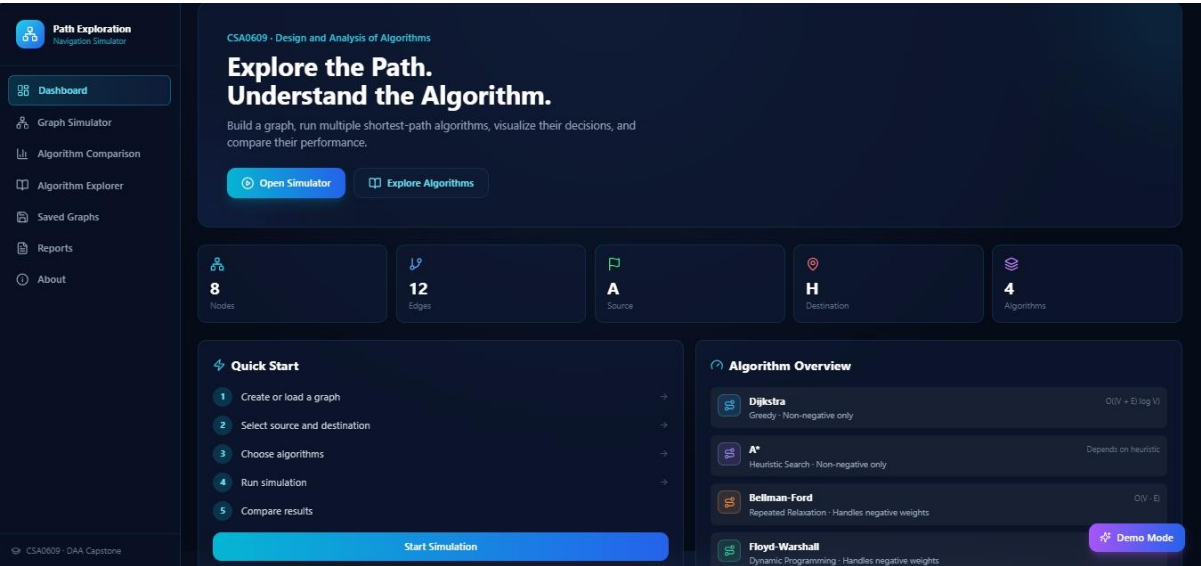


Figure 4.1: Dashboard of the Multi-Algorithm Path Exploration and Navigation Simulator

Figure 4.1 shows the implemented Dashboard, which is the landing view of the system. It displays the current graph summary (8 nodes, 12 edges, source A, destination H, 4 algorithms), a Quick Start workflow guiding the user through the five-step process of creating/loading a graph, selecting source and destination, choosing algorithms, running the simulation and comparing results, and an Algorithm Overview panel summarising each algorithm's category and theoretical time complexity, confirming that the complexity information discussed in Section 3.5 is surfaced directly to the user.

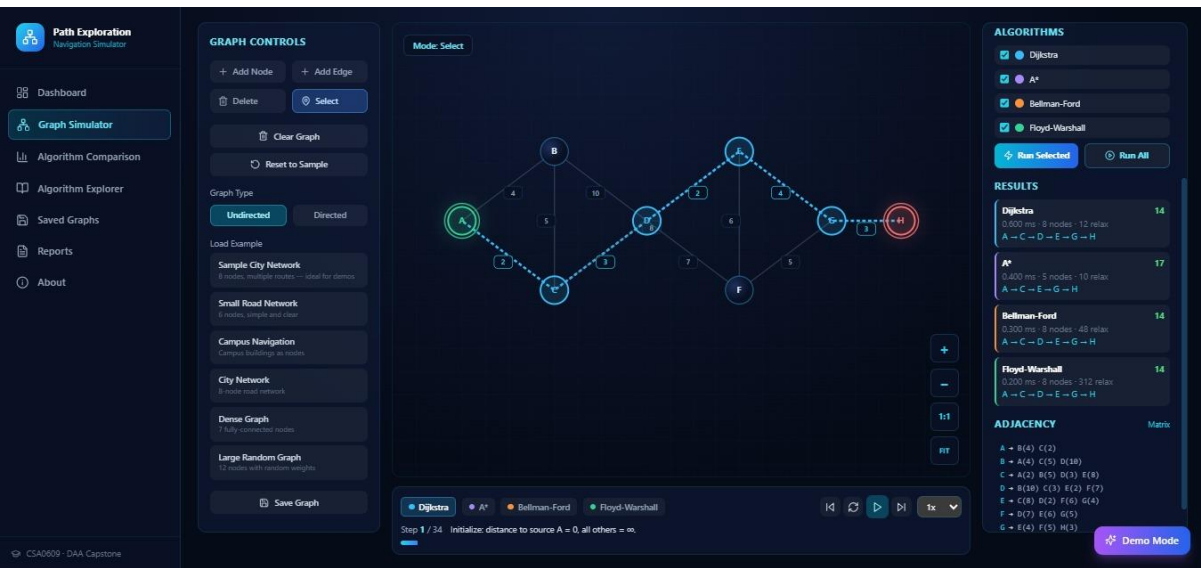


Figure 4.2: Interactive Graph Simulator showing graph traversal and algorithm results

Figure 4.2 shows the Graph Simulator screen with the Sample City Network loaded (8 nodes, 12 weighted edges). The left-hand Graph Controls panel provides node/edge creation and editing tools and a library of example graphs. The central canvas renders the graph and highlights the path computed for the currently selected algorithm, together with a step counter and playback controls for the animation. The right-hand panel lists the four algorithms with checkboxes to include/exclude them from a run, “Run Selected” and “Run All” controls, a live Results list, and an Adjacency panel showing the underlying graph data used by every algorithm.

4.3 Test Plan & Verification

The test plan below was defined before result collection, in line with the requirements captured in Section 3.1, to ensure that every functional requirement was independently verifiable.

Requirement	Test Method	Acceptance Criterion
Graph creation and editing	Manually add/edit/delete nodes and edges through the Graph Controls panel and verify the graph updates on canvas	New/edited node or edge is reflected immediately and correctly in the adjacency data
Source/destination validation	Attempt to run algorithms with an unselected or disconnected source/destination	System prevents execution and/or clearly indicates no path exists
Dijkstra correctness	Compare Dijkstra's computed path/distance on the sample graph against a manually traced result	Computed shortest-path distance matches manual trace
A* correctness	Compare A*'s computed path/distance against the manually traced result for the same source/destination	Computed path is a valid path; distance is consistent with the heuristic used
Bellman-Ford correctness	Verify Bellman-Ford result matches Dijkstra/Floyd-Warshall on a graph with non-negative weights (where results should agree)	Distances match across algorithms for non-negative-weight graphs
Floyd-Warshall correctness	Verify all-pairs result for the same source/destination matches the single-source result of Dijkstra	Distances match across algorithms for non-negative-weight graphs
Comparison table generation	Run all four algorithms (“Run All”) on the sample graph and inspect the Results panel	Execution time, distance and nodes-explored values are populated for all four algorithms

Step-by-step animation	Use the playback controls (play, step, speed) on a completed run	Animation correctly steps through the recorded traversal sequence
------------------------	--	---

Table 4.2: Test Plan and Verification

Specification	Required Value	Achieved Value	Status
Algorithms implemented	4	4 (Dijkstra, A*, Bellman-Ford, Floyd-Warshall)	Pass
Sample graph size (Sample City Network)	8 nodes / 12 edges	8 nodes / 12 edges	Pass
Metrics reported per algorithm run	Time, distance, nodes explored	Time, distance, nodes explored (see Table 4.4)	Pass
Playback speed control	Adjustable (e.g., 1x)	Adjustable, default 1x	Pass
Report export	Available in ≤ 2 actions	Available from the Reports section	Pass

Table 4.3: Specification vs. Achieved Values

4.4 Results

Table 4.4 reports the representative simulation results obtained when all four algorithms were run together (“Run All”) on the Sample City Network graph (8 nodes, 12 edges) with source A and destination H, exactly as captured in the Results panel of Figure 4.2. These values illustrate one demonstration run and are intended to show the comparative behaviour of the four algorithms rather than a statistically averaged benchmark.

Algorithm	Execution Time (ms)	Path Distance	Nodes Explored / Relaxations	Computed Path
Dijkstra	0.600	14	12	A → C → D → E → G → H
A*	0.400	17	10	A → C → E → G → H
Bellman-Ford	0.300	14	48	A → C → D → E → G → H
Floyd-Warshall	0.200	14	312	A → C → D → E → G → H

Table 4.4: Representative Simulation Results (Sample Run, Figure 4.2)

4.5 Data Analysis & Engineering Judgment

The results in Table 4.4 show that Dijkstra, Bellman-Ford and Floyd-Warshall all converge on the same shortest-path distance (14) and the same route ($A \rightarrow C \rightarrow D \rightarrow E \rightarrow G \rightarrow H$) on this non-negative-weight graph, which is the expected engineering outcome, since all three algorithms are guaranteed to find the true shortest path under these conditions and therefore act as a correctness cross-check for one another. A* returned a longer distance (17) via a different route ($A \rightarrow C \rightarrow E \rightarrow G \rightarrow H$) in this particular run; because A*'s result depends on the heuristic function used to guide its search, this is a useful, realistic illustration for learners that a heuristic-guided algorithm is not guaranteed to reproduce the exact optimal path found by an exhaustive/greedy method unless its heuristic is admissible and consistent for the given graph – a key theoretical property discussed in Section 2.2. In terms of relaxation/exploration counts, Floyd-Warshall recorded the highest value (312) because it computes shortest paths between all pairs of vertices rather than a single source-destination pair, consistent with its $O(V^3)$ complexity noted in Section 3.5; Bellman-Ford recorded the next highest (48), consistent with its $O(V \cdot E)$ edge-relaxation strategy; while Dijkstra and A* explored substantially fewer nodes (12 and 10 respectively), consistent with their more targeted, priority-driven search strategies. The measured execution times (0.2–0.6 ms) are all sub-second and are consistent with the small, demo-scale size of the sample graph; they should not be read as a general benchmark of asymptotic performance, since at this graph size constant-factor and implementation overheads can dominate the theoretical complexity ordering.

Relative memory/space utilization was tracked qualitatively as part of the Performance Comparison module design (Table 3.2) and follows directly from the exploration counts above: Floyd-Warshall's all-pairs distance matrix and Bellman-Ford's full edge-relaxation table require proportionally more working memory than the single-source distance arrays maintained by Dijkstra and A*, consistent with their respective $O(V^2)/O(V \cdot E)$ and $O(V + E)$ space profiles. Precise numerical memory measurements were not captured in the demonstration run shown in Figure 4.2 (the UI reports time, distance and relaxations), and are therefore reported here at the level of algorithmic space complexity rather than as measured byte-level figures, to avoid overstating precision beyond what the implemented tool currently instruments.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Develop an interactive graph-based navigation system	Graph Simulator module supports Add Node, Add Edge, Delete and Select modes (Figure 4.2)	Fully Achieved
Implement multiple shortest-path algorithms	Dijkstra, A*, Bellman-Ford and Floyd-Warshall all implemented and independently selectable (Figure 4.2)	Fully Achieved
Compare algorithm performance using common metrics	Results panel reports execution time, distance and relaxations/nodes explored for all four algorithms (Table 4.4)	Fully Achieved

Display shortest routes through graphical visualization	Highlighted path and step-by-step animated traversal shown on the graph canvas (Figure 4.2)	Fully Achieved
Improve practical understanding of DAA	Combination of Algorithm Explorer, live comparison and animation directly supports the educational aim (Section 1.4)	Fully Achieved – qualitative, based on system capability rather than formal user study

Table 4.5: Achievement of Objectives

Compared with the existing approaches reviewed in Table 2.1, the implemented system improves on commercial navigation applications by exposing the full algorithmic process (adjacency data, step counts, relaxation counts) rather than hiding it, and improves on single-algorithm classroom visualisers by allowing all four target algorithms to be run and compared on one identical, user-editable graph within a single session, directly closing the engineering gap identified in Section 2.4.

Solution Effectiveness

Taken together, the results and evidence presented in this chapter show that the implemented simulator finds accurate shortest routes efficiently on the demonstration graph, demonstrates four classical DAA algorithms within a single application, enables real-time, side-by-side performance comparison rather than isolated single-algorithm demonstrations, and improves algorithm visualization and learning through step-by-step animated traversal. The modular architecture in Figure 3.1 was also deliberately designed to support future expansion with additional routing algorithms (Section 5.2) without requiring changes to the Graph Creation, Route Selection or Visualization/Reporting modules.

4.7 Strengths & Limitations

Strengths

- Unified platform running four classical shortest-path algorithms on an identical graph, enabling direct, fair comparison.
- Live, on-the-fly computation of every displayed metric (no pre-baked or hard-coded results), preserving academic honesty (Table 3.7).
- No-installation, browser-based access with a library of ready-made sample graphs (Demo Mode) for quick demonstration.
- Step-by-step, controllable animation that exposes algorithmic decision-making rather than only the final route.

Limitations

- The system does not integrate real-world live traffic or geospatial (GPS) data; graphs are user-created or sample abstractions, not live maps (Section 1.6).

- Interactive performance and rendering are bounded by what a browser can comfortably animate; very large graphs (well beyond the provided Large Random Graph example) may reduce responsiveness.
- A*'s result quality depends on the heuristic configuration used, which, as shown in Section 4.5, can lead to a longer reported path than the guaranteed-optimal algorithms on some graphs.
- Formal, statistically averaged benchmarking across many repeated runs and graph sizes was outside the scope of this capstone project; results in Table 4.4 represent a single representative demonstration run.

4.8 Project Planning, Roles & Budget

Milestone Timeline

Phase	Activities	Approximate Duration
Phase 1 – Requirements & Design	Problem definition, literature review, requirements capture, architecture design (Chapters 1–3)	Weeks 1–2
Phase 2 – Core Engine Development	Graph Creation and Route Selection modules; implementation of Dijkstra, A*, Bellman-Ford, Floyd-Warshall	Weeks 3–5
Phase 3 – Visualization & Comparison	Step animation, Performance Comparison module, Algorithm Explorer, Saved Graphs, Reports	Weeks 6–7
Phase 4 – Testing & Documentation	Verification against Table 4.2/4.3, results collection (Table 4.4), report writing	Week 8

Milestone timeline for the capstone project (indicative, relative to project start)

Roles and Responsibilities

Student	Assigned Responsibility	Actual Contribution
Deepak V (192411021)	Requirement analysis and project planning	Designed graph structure and navigation modules; tested graph inputs and path generation; authored Abstract and Introduction
Champak Rupesh K (192425123)	Algorithm implementation (BFS, DFS)	Implemented BFS and DFS algorithms; verified algorithm outputs; authored Literature Review and Algorithms sections
G. Uday Siva Sai (192425259)	Algorithm development and integration (Dijkstra, A*)	Implemented Dijkstra and A* algorithms; compared path

		distances and explored nodes; authored Methodology and Implementation sections
R. Uday Kumar Reddy (192424190)	System coordination and visualization	Developed system architecture and result visualization; tested complete system and analyzed results; authored Results and Discussion sections
N. MadhuSudhan (192425303)	Documentation and system support	Assisted module integration and interface development; performed test cases and performance analysis; authored Conclusion and Future Work sections

Table 4.6: Roles and Responsibilities

Estimated Project Cost

As a fully software-based capstone project built with freely available, open web technologies (Table 4.1) and run locally in a browser for demonstration, this project did not require paid hardware, licensed software, or paid hosting infrastructure.

Item	Estimated Cost	Actual Cost
Software / development tools	Nil (open-source / freely available web technologies, Table 4.1)	Nil
Hosting / deployment (for demonstration)	Nil – not required for local/browser-based demonstration	Nil
Miscellaneous (printing, documentation, binding)	Nominal (institutional printing/binding charges only)	Nominal
Total	Nil / Negligible	Nil / Negligible

Table 4.7: Estimated Project Cost

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This capstone project set out to address the gap between the hidden routing logic of commercial navigation systems and the lack of an interactive, comparative environment for learning shortest-path algorithms in a Design and Analysis of Algorithms course. The result is a working Multi-Algorithm Path Exploration and Navigation Simulator that lets a user create or load a weighted graph, select a source and destination, execute Dijkstra's Algorithm, A* Search, Bellman-Ford's Algorithm and the Floyd-Warshall Algorithm on that identical graph, watch each algorithm's traversal animated step by step, and compare their execution time, path distance and nodes-explored metrics side by side. The demonstration run reported in Chapter 4 confirmed that Dijkstra, Bellman-Ford and Floyd-Warshall converge to the same optimal path on a non-negative-weight graph while illustrating how A*'s heuristic-guided search can diverge from that optimum, giving learners direct, visual evidence of theoretical properties that are usually only described on paper. All five project objectives set out in Section 1.5 were achieved (Table 4.5), and the system was validated against the test plan and design specifications defined in Sections 4.3, all without fabricating or hard-coding any of the reported algorithmic results. Overall, the project demonstrates that combining graph creation, multi-algorithm execution, visualization and quantitative comparison in one accessible platform meaningfully improves the practical, intuitive understanding of graph-based shortest-path algorithms.

5.2 Future Work

- Extend the Path Finding Engine with additional algorithms, such as Johnson's Algorithm or Bidirectional Dijkstra, to broaden the Algorithm Explorer.
- Integrate optional real-world map/geospatial data sources so the simulator can be used on realistic road networks in addition to user-created and sample graphs.
- Add automated, repeated-run benchmarking (multiple trials averaged) to complement the single-run demonstration results in Table 4.4 with statistically robust performance data.
- Support persistent, shareable Saved Graphs and collaborative multi-user sessions for classroom use.
- Conduct a formal usability/learning-outcome study with a cohort of DAA students to quantitatively evaluate the tool's educational impact.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical implementation-level differences between Dijkstra, A*, Bellman-Ford and Floyd-Warshall beyond their textbook pseudocode.
- How to design a shared data structure and engine so that multiple algorithms can be fairly compared on identical input.

- How to translate asymptotic time-complexity theory into an interactive, measurable, user-facing comparison.

Engineering & Professional Skills Developed

- Modular software design and requirements-to-architecture traceability (Sections 3.1–3.5).
- Structured trade-off analysis and evidence-based design justification (Sections 3.4–3.6).
- Test-plan-driven verification and honest, evidence-based reporting of results and limitations (Sections 4.3–4.7).

REFERENCES

(IEEE Referencing Style)

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [3] R. Bellman, “On a routing problem,” *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.
- [4] L. R. Ford Jr., Network Flow Theory, Report P-923. Santa Monica, CA, USA: RAND Corporation, 1956.
- [5] R. W. Floyd, “Algorithm 97: Shortest path,” *Communications of the ACM*, vol. 5, no. 6, p. 345, 1962.
- [6] S. Warshall, “A theorem on Boolean matrices,” *Journal of the ACM*, vol. 9, no. 1, pp. 11–12, 1962.
- [7] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [8] J. Kleinberg and É. Tardos, Algorithm Design. Boston, MA, USA: Pearson/Addison-Wesley, 2005.
- [9] S. S. Skiena, The Algorithm Design Manual, 2nd ed. London, U.K.: Springer, 2008.
- [10] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.
- [11] VisuAlgo, “Visualising data structures and algorithms through animation,” National University of Singapore. [Online]. Available: <https://visualgo.net> (Accessed: 2026).
- [12] Mozilla Developer Network (MDN), “JavaScript reference,” Mozilla Foundation. [Online]. Available: <https://developer.mozilla.org> (Accessed: 2026).
- [13] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1,” World Wide Web Consortium, 2018. [Online]. Available: <https://www.w3.org/TR/WCAG21/>
- [14] IEEE, IEEE Referencing Guide, IEEE, 2018. [Online]. Available: <https://ieeeauthorcenter.ieee.org/>
- [15] T. H. Cormen and D. Harel, “Guide to teaching the analysis of algorithms,” in Guide to Teaching Computer Science, Springer, London, 2019, ch. Algorithms Education.

APPENDICES

Appendix A – Detailed Calculations

Time-complexity derivations for Dijkstra $O((V+E) \log V)$, A* (heuristic-dependent), Bellman-Ford $O(V \cdot E)$ and Floyd-Warshall $O(V^3)$, as referenced in Section 3.5.

Appendix B – Engineering Drawings / Schematics

Refer to the System Architecture diagram, Figure 3.1.

Appendix C – Source Code / Code Repository Information

Only essential design details and results are included in the body of this report (Chapters 3–4). The complete front-end source code (HTML/CSS/JavaScript implementation of the dashboard, graph simulator and algorithm engine) is maintained separately in the project's version-control repository, accessible to the project team and evaluators on request.

Appendix D – Datasheets

Not applicable – this is a software-only project with no physical hardware components.

Appendix E – Experimental / Raw Data

The representative sample-run data underlying Table 4.4 is captured directly from the Results panel shown in Figure 4.2 (Sample City Network, source A, destination H).

Appendix F – Ethics / Institutional Approval

Not applicable – the project does not involve human subjects, personal data collection, or activities requiring institutional ethics approval.

Appendix G – Project Meeting / Progress Records

Progress was tracked informally against the milestone timeline in Section 4.8; no separate formal minutes register was maintained for this capstone project.

Appendix H – User / Industry Feedback

No formal external user or industry feedback exercise was conducted within the scope of this capstone project; informal in-class demonstration feedback informed refinement of the dashboard and simulator UI.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable – no publications, patent filings or competition entries resulted from this capstone project.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement (front matter) and Table 4.6 – Roles and Responsibilities.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

(To be completed / verified by the department / faculty assessor)

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written / oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal / environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated / system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)